



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Writing Spark Dataframes

Requirement:

In the previous lecture, we saw the following requirement.

We have a JSON data set, and you are asked to load this dataframe and create a Spark table. For your reference, here is one sample record, and here are details about the columns shown below.

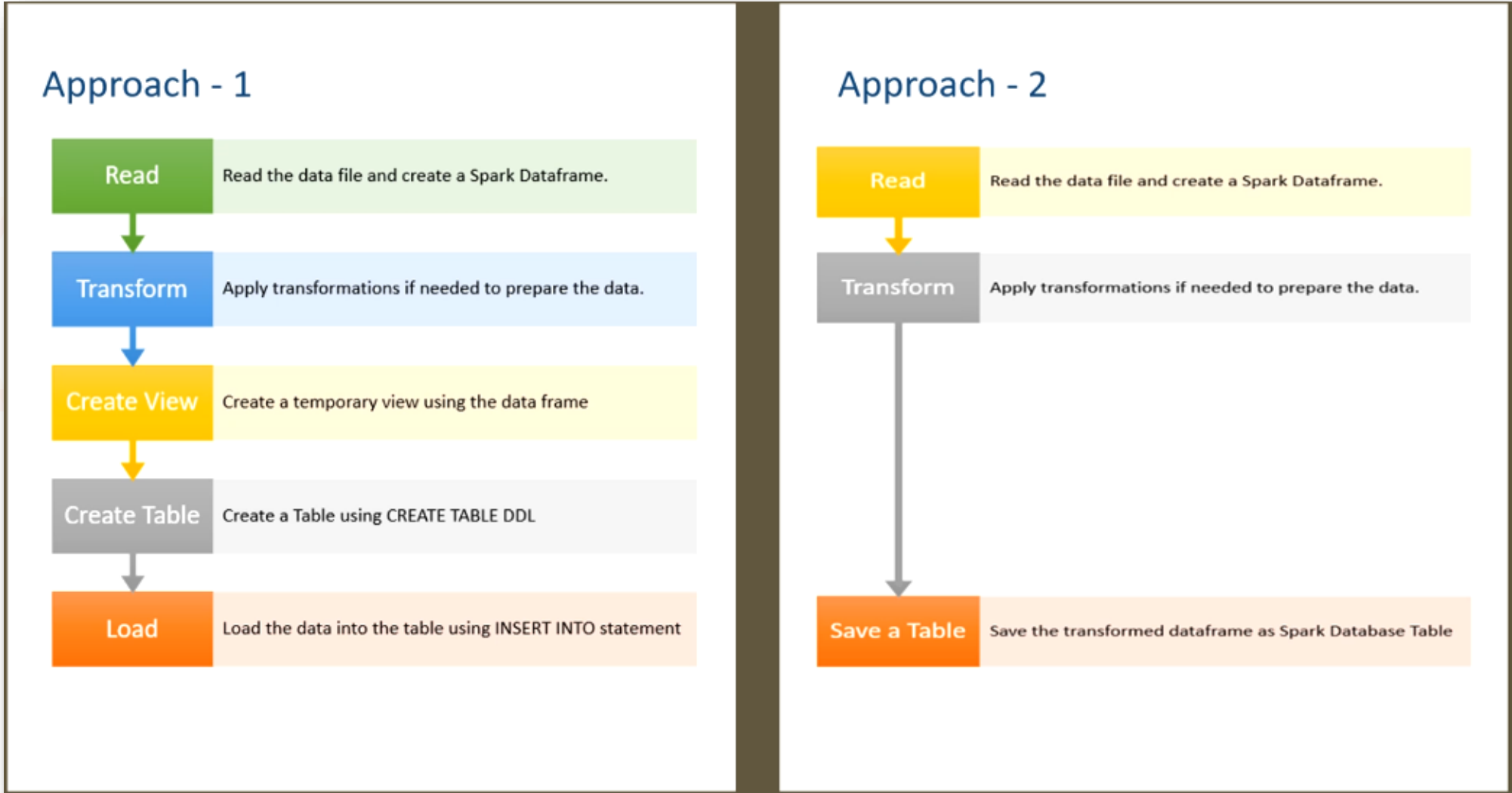
Flight Delay Data

```
{
  "FL_DATE": "1/1/2000",
  "OP_CARRIER": "DL",
  "OP_CARRIER_FL_NUM": 1451,
  "ORIGIN": "BOS",
  "ORIGIN_CITY_NAME": "Boston, MA",
  "DEST": "ATL",
  "DEST_CITY_NAME": "Atlanta, GA",
  "CRS_DEP_TIME": 1115,
  "DEP_TIME": 1113,
  "WHEELS_ON": 1343,
  "TAXI_IN": 5,
  "CRS_ARR_TIME": 1400,
  "ARR_TIME": 1348,
  "CANCELLED": 0,
  "DISTANCE": 946
}
```

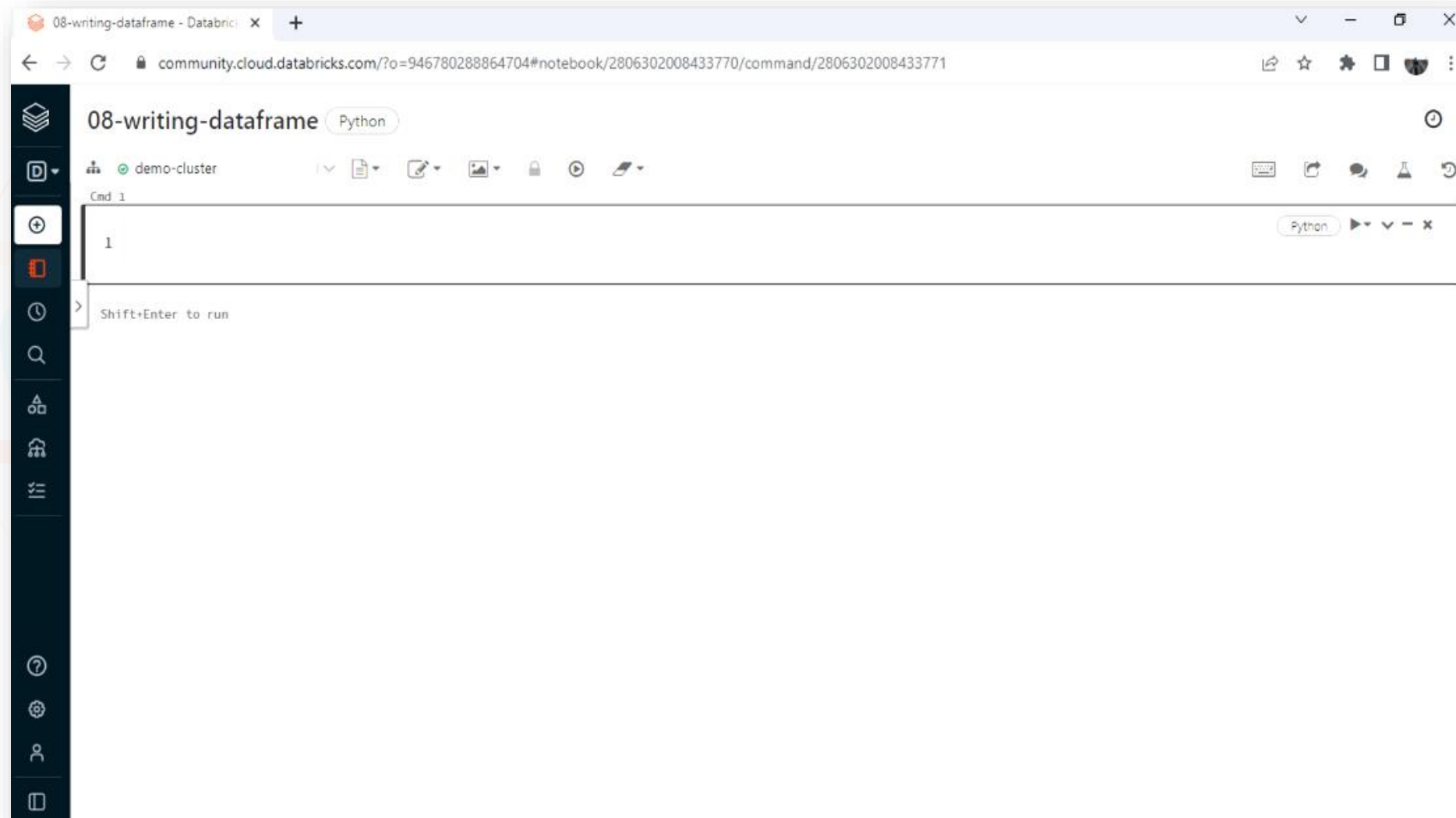
1. Flight date in MM/DD/YYYY format
2. Unique carrier code
3. Flight number
4. Origin IATA Airport Code
5. Origin City Name
6. Destination IATA Airport Code
7. Destination City Name
8. Scheduled Departure time in HHMM
9. Actual Departure time in HHMM
10. Flight Duration in HHMM
11. TaxiIn taxi in time, in minutes
12. Scheduled Arrival time in HHMM
13. Actual Departure time in HHMM
14. Was it canceled? 1 for a true, and 0 is false
15. Distance in miles

In order to serve our requirement, we have the following two approaches.

We saw the first approach in the previous documentation, now let us see the second approach in this documentation.



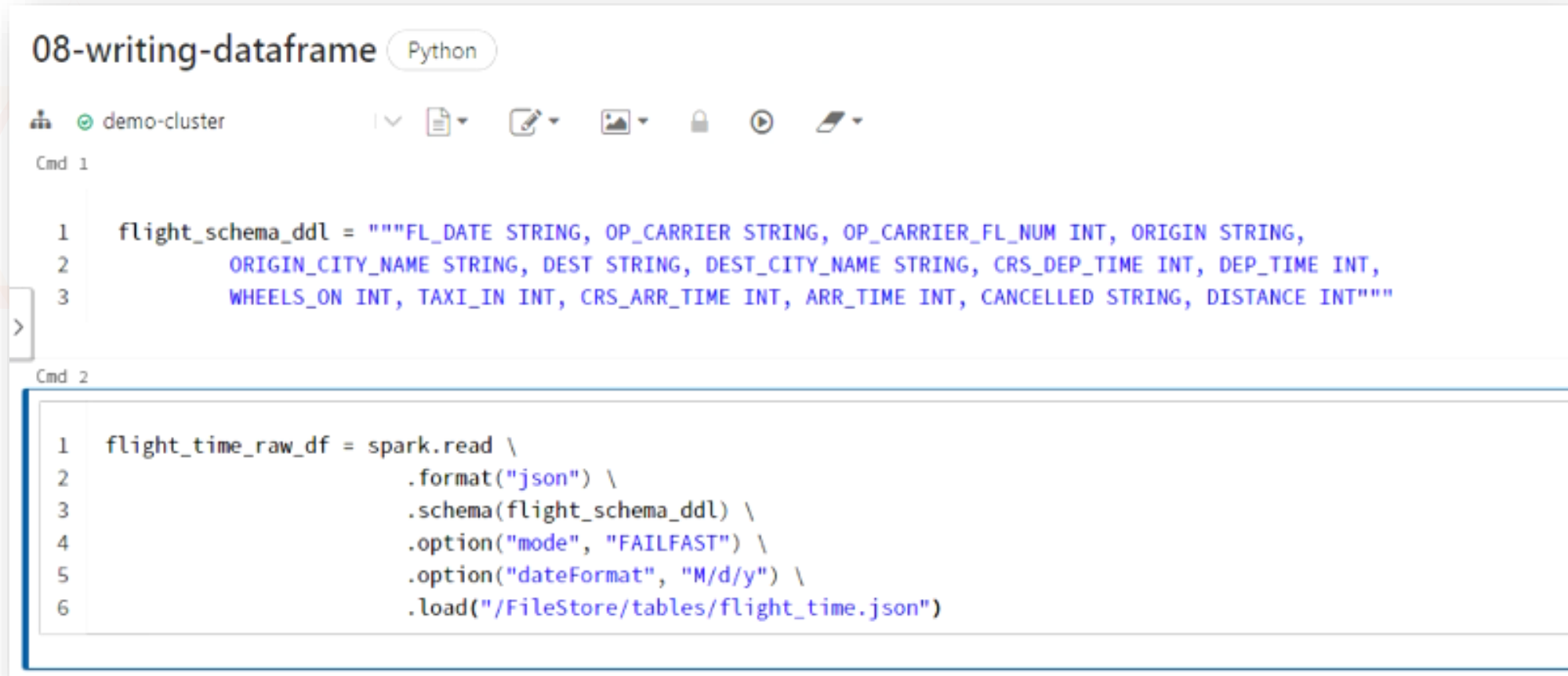
Go to your Databricks Workspace and create a new notebook. **(Reference - 08-writing-dataframe.ipynb)**



We already wrote the code for the following:

1. Define a flight delay dataset schema.
2. Load data from the file and create a Dataframe.

So I have copy-pasted the code here.



```
08-writing-dataframe Python

demo-cluster

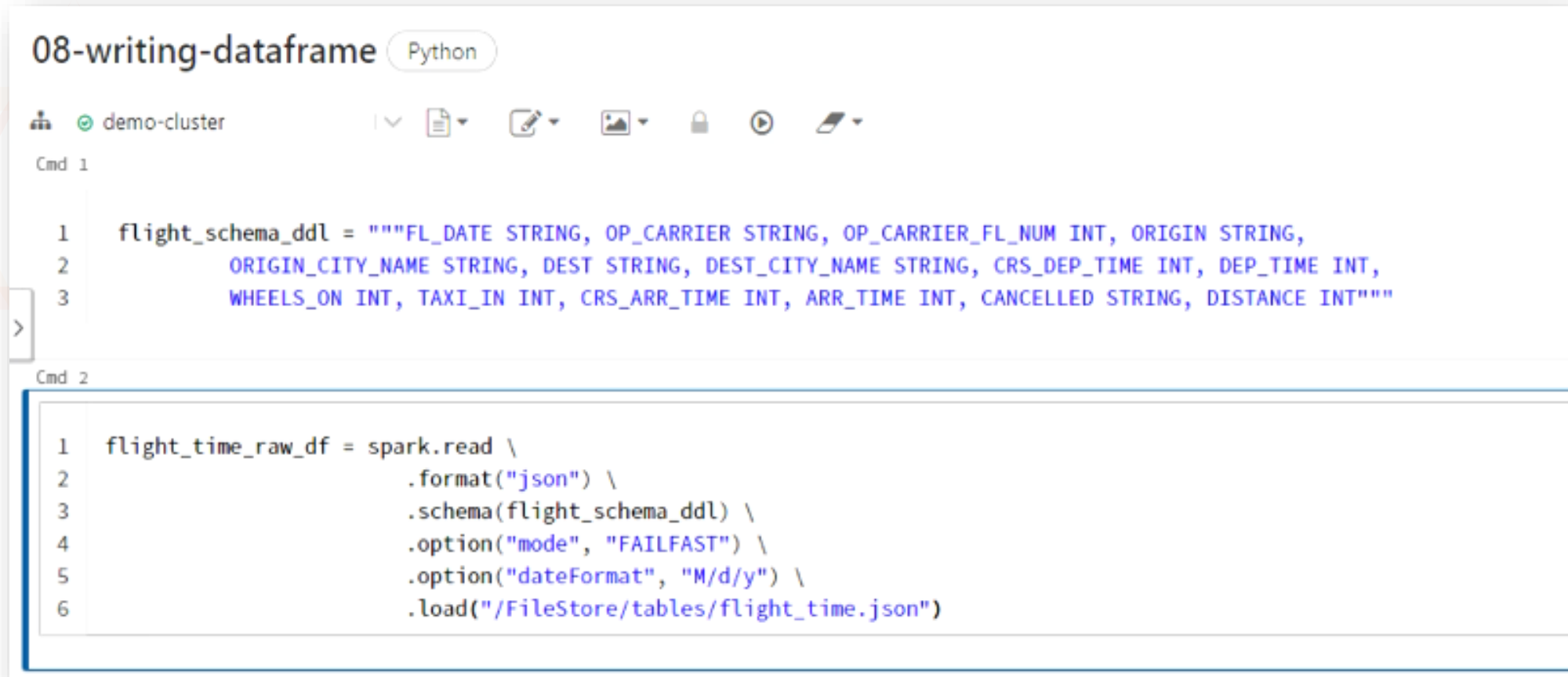
Cmd 1

1 flight_schema_ddl = """FL_DATE STRING, OP_CARRIER STRING, OP_CARRIER_FL_NUM INT, ORIGIN STRING,
2   ORIGIN_CITY_NAME STRING, DEST STRING, DEST_CITY_NAME STRING, CRS_DEP_TIME INT, DEP_TIME INT,
3   WHEELS_ON INT, TAXI_IN INT, CRS_ARR_TIME INT, ARR_TIME INT, CANCELLED STRING, DISTANCE INT"""

Cmd 2

1 flight_time_raw_df = spark.read \
2   .format("json") \
3   .schema(flight_schema_ddl) \
4   .option("mode", "FAILFAST") \
5   .option("dateFormat", "M/d/y") \
6   .load("/FileStore/tables/flight_time.json")
```

So we have this `flight_time_raw_df`. And I want to save this dataframe into a Spark table. That is my requirement.
But before saving this data as a Spark table, I want to correct the `FL_DATE` and `CANCELLED` data types.



The screenshot shows a Databricks notebook interface. The title bar indicates the notebook is named '08-writing-dataframe' and is running Python code. Below the title bar, there is a toolbar with icons for various actions like running, saving, and viewing. The main area of the notebook contains two command blocks. The first block, labeled 'Cmd 1', contains a single line of Python code that defines a schema string. The second block, labeled 'Cmd 2', contains a multi-line Python code snippet that reads a DataFrame from a JSON file and writes it to a Spark table, specifying various options like format, schema, mode, and date format.

```
08-writing-dataframe Python

demo-cluster

Cmd 1

1 flight_schema_ddl = """FL_DATE STRING, OP_CARRIER STRING, OP_CARRIER_FL_NUM INT, ORIGIN STRING,
2   ORIGIN_CITY_NAME STRING, DEST STRING, DEST_CITY_NAME STRING, CRS_DEP_TIME INT, DEP_TIME INT,
3   WHEELS_ON INT, TAXI_IN INT, CRS_ARR_TIME INT, ARR_TIME INT, CANCELLED STRING, DISTANCE INT"""

Cmd 2

1 flight_time_raw_df = spark.read \
2   .format("json") \
3   .schema(flight_schema_ddl) \
4   .option("mode", "FAILFAST") \
5   .option("dateFormat", "M/d/y") \
6   .load("/FileStore/tables/flight_time.json")
```

We have already learned about the following:

1. withColumn() transformations
2. to_date() function

In fact, we converted a string field to a date field in an earlier lecture. So I guess you can fix the FL_DATE quickly.

But how do you fix the CANCELLED?

The idea is to look for something that helps you write the following logic:

if CANCELLED == 1 then true else false

Functions in Spark are defined in two places.

1. pyspark.sql.functions package
2. Built-In Spark SQL functions

You can check for the Built-In SQL functions in the documentation, and you will see an if function, and it takes three arguments.

The first argument is an expression that must evaluate as true or false.

The other two arguments are true or false.

I can use this function in my Dataframe code to transform the CANCELLED column from a number to a boolean.

Here is the code to transform the FL_DATE and CANCELLED columns.

I am importing to_date() and expr() functions because I need them. Then I am starting with flight_time_raw_df and transforming two columns using the withColumn() transformation.

The withColumn() takes two arguments. The first argument is the column's name that you want to transform. The second argument is the transformation logic. The logic for FL_DATE is as simple as applying the to_date() function, so we change it from a string to a date.

The logic for the CANCELLED column is also straightforward. We want to apply the if() built-in function as shown here. If CANCELLED is one, make it True else, make it False. Also, my logic for transforming the CANCELLED column is a SQL expression. It is not a dataframe expression. So I must explicitly evaluate the SQL expression. And that's why we use the expr() function.

Cmd 3

```
1 from pyspark.sql.functions import to_date, expr
2
3 flight_time_df = flight_time_raw_df \
4     .withColumn("FL_DATE", to_date("FL_DATE", "M/d/y")) \
5     .withColumn("CANCELLED", expr("if(CANCELLED=1, true, false)"))
```

Command took 0.09 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:05:17 AM on demo-cluster

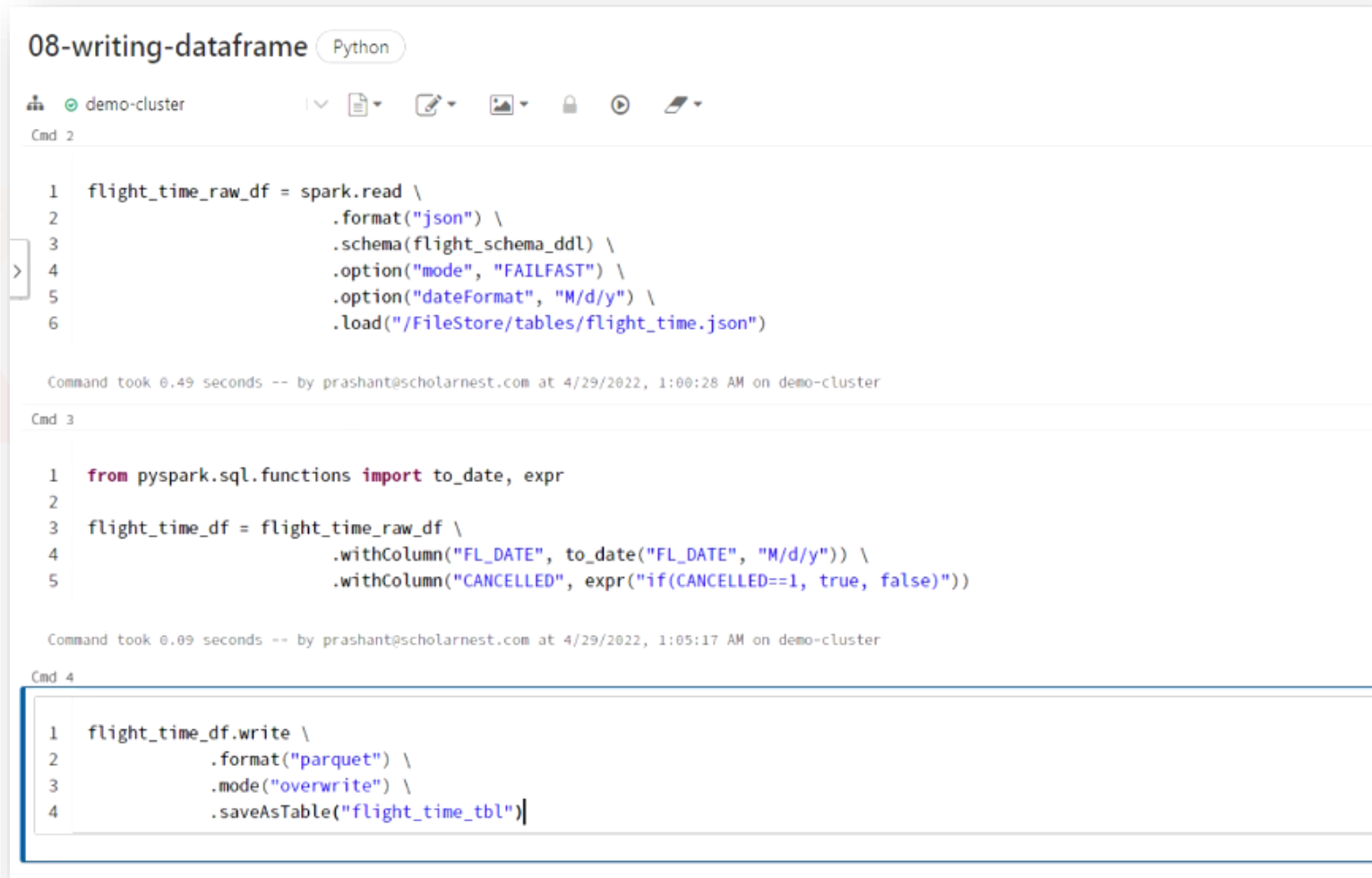
Cmd 4

You can save your Dataframe into a database table using the code shown below.

Cmd 4

```
1 flight_time_df.write \  
2     .format("parquet") \  
3     .mode("overwrite") \  
4     .saveAsTable("flight_time_tbl")|
```

If you look at the code, it looks similar to the DataFrameReader. So you can see the spark.read here. And you can see the dataframe.write here. The first code block is reading data from a source and creating a Dataframe. The second code block is from writing Dataframe to a target. The first one starts with a spark session and access the DataFrameReader using the read attribute. The second one starts with a Dataframe and access the DataFrameWriter using the write attribute. Once you have a DataFrameWriter, the rest of the code structure is the same as the DataFrameReader code. We can set the format, set some options, and save it.

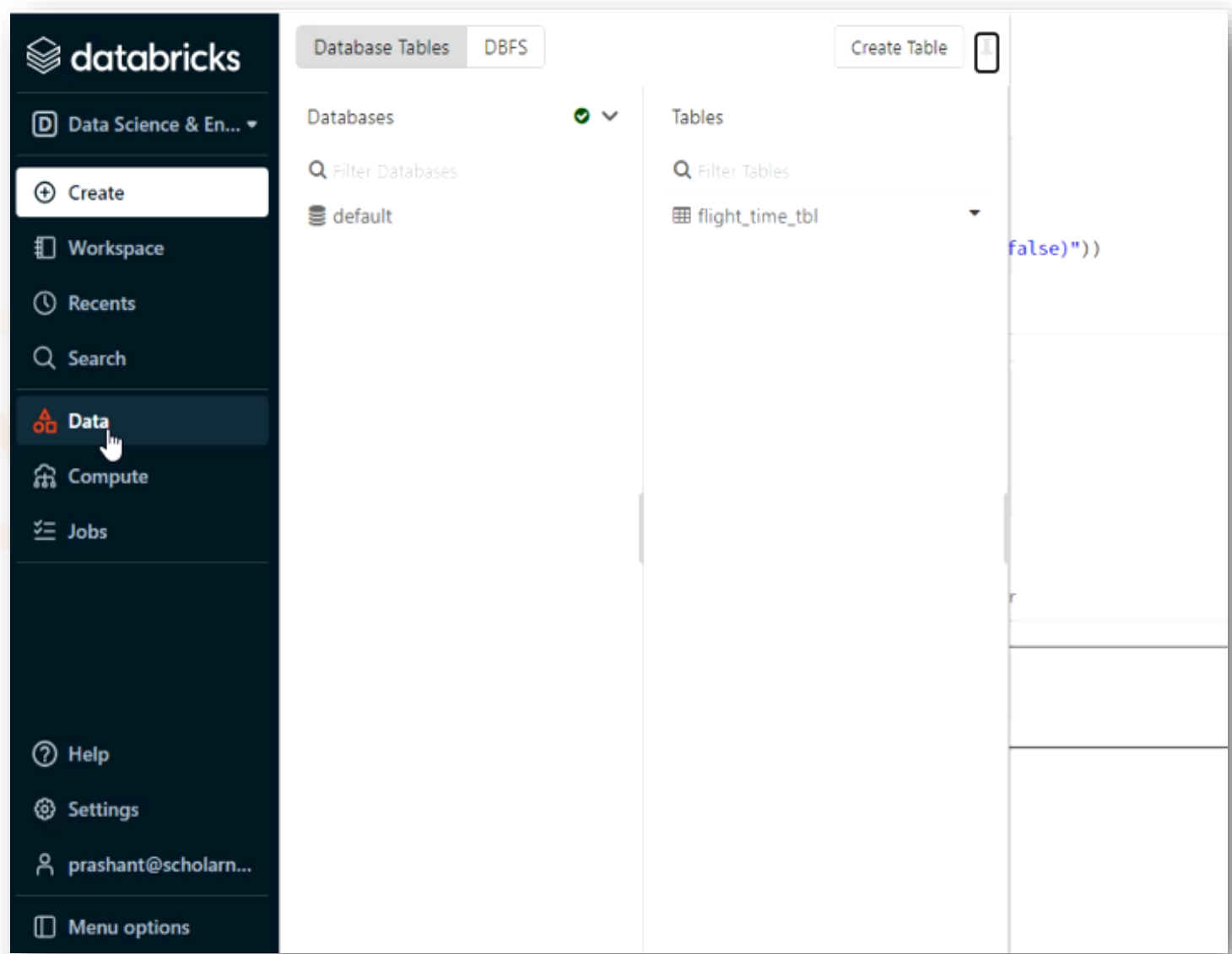


```
08-writing-dataframe Python
demo-cluster
Cmd 2
1 flight_time_raw_df = spark.read \
2     .format("json") \
3     .schema(flight_schema_ddl) \
4     .option("mode", "FAILFAST") \
5     .option("dateFormat", "M/d/y") \
6     .load("/FileStore/tables/flight_time.json")
Command took 0.49 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:00:28 AM on demo-cluster
Cmd 3
1 from pyspark.sql.functions import to_date, expr
2
3 flight_time_df = flight_time_raw_df \
4     .withColumn("FL_DATE", to_date("FL_DATE", "M/d/y")) \
5     .withColumn("CANCELLED", expr("if(CANCELLED==1, true, false)))
Command took 0.09 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:05:17 AM on demo-cluster
Cmd 4
1 flight_time_df.write \
2     .format("parquet") \
3     .mode("overwrite") \
4     .saveAsTable("flight_time_tbl")
```

DataFrameWriter supports the following modes:

1. append: Append contents of this DataFrame to existing data.
2. overwrite: Overwrite existing data.
3. error or errorifexists: Throw an exception if data already exists.
4. ignore: Silently ignore this operation if data already exists.

If you look at the data section, you will see a table is created for your Dataframe.



You can also describe the table from your notebook as shown below.
And you will see that both FL_DATE and CANCELLED columns are rectified as per your requirement.

1 %sql

2 describe extended flight_time_tbl

	col_name	data_type	comment
1	FL_DATE	date	null
2	OP_CARRIER	string	null
3	OP_CARRIER_FL_NUM	int	null
4	ORIGIN	string	null
5	ORIGIN_CITY_NAME	string	null
6	DEST	string	null
7	DEST_CITY_NAME	string	null

Showing all 30 rows.

Command took 0.48 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:09:48 AM on demo-cluster



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com